

1. Design an algorithm for 2 Dimensional Matrix Addition Compute its Time complexity using frequency count method, also trace the algorithm, for the array of size 3 x 4.

Solution:

```
algorithm matrix_addition(A, B, R, C)

if A.rows != B.rows OR A.cols != B.cols: -----2
    return "Matrices must have the same dimensions for addition."-----2

C = new R x C matrix -----1

for i from 0 to R-1:----- R
    // Iterate through columns
    for j from 0 to C-1:-----R * C
        // Add corresponding elements
        C[i][j] = A[i][j] + B[i][j] -----R * C

return C ----- 1

Total = 2+2+1 + R + 2RC + 1 ==>6 + R + 2RC
Dominant Term is 2RC => O(RC) ==> O(n^2)
```

2. Explain the process of implementing Linear Search using arrays. Trace the algorithm to search an element 32 from the list of elements: 12, 27, 19, 32, 45.

Solution: Linear search is a sequential search

Trace to Search for 32

Given Array: [12, 27, 19, 32, 45]

The following is a step-by-step trace of the linear search algorithm.

Step	Index (i)	Array Element (array[i])	Comparison (array[i] == 32)	Outcome
1	0	12	12 == 32 is False	The element does not match. Continue to the next element.
2	1	27	27 == 32 is False	The element does not match. Continue to the next element.
3	2	19	19 == 32 is False	The element does not match. Continue to the next element.
4	3	32	32 == 32 is True	A match is found at index 3. Stop the search.

The required number is at index 3;

3. What is the smallest value of n such that an algorithm whose running time is $100n^2$ runs faster than an algorithm whose running time is 2^n on the same machine?

Solution:

We evaluate both functions, $100n^2$ and 2^n for increasing values of n .

n	$100n^2$	2^n	$100n^2 < 2^n$
1	$100(1)^2 = 100$	$2^1 = 2$	No
2	$100(2)^2 = 400$	$2^2 = 4$	No
5	$100(5)^2 = 2,500$	$2^5 = 32$	No
10	$100(10)^2 = 10,000$	$2^{10} = 1,024$	No
14	$100(14)^2 = 19,600$	$2^{14} = 16,384$	No
15	$100(15)^2 = 22,500$	$2^{15} = 32,768$	Yes

The smallest value $n = 15$ at which $100n^2$ will be less than 2^n

4) Find the time complexity of the following algorithm. Note: Must write increasing/decreasing pattern of algorithms.

a)

Solution

Sum = 0 -----1

```
For (i=n; i>=1 ; i=i/2)
    sum=sum+i;
printf("Sum = %d", sum)
```

Here there is a for loop with dividing by constant 2, till n^2

Here is the dividing pattern

$n^2, n^2/2, n^2/2^2, n^2/2^2, \dots, n^2/2^k$

Assume $n^2/2^k = 1$ so $n^2 = 2^k$

Log2 on Both Side

$$\log_2 2^k = \log_2 n^2$$

$$K = 2 \log n$$

O(Logn) (Loop running time)

Here following statements are inside the loop

```
sum=sum+l;
printf("Sum = %d", sum)
```

and sum=0 take O(1)

So Now it will be $2 \log n + 1$, after eliminating constants

O(Logn)

b)

Solution:

Here there is Independent Nested Loop

But there are two inner loops,

2nd Inner Loop for(k=1; k<p; k=k*2) depends on 1st inner loop for(j=n; j>1; j=j/2) due to P.

Outer Loop for(i=1; i<n ; i++) runs n-1 times, **O(n)**

1st Inner Loop for(j=n; j>1; j=j/2) ----- **Logn times** (b/c dividing by constant , here P is increasing by 1 in each iteration (++p))

2nd Inner Loop for(k=1; k<p; k=k*2) is running until k<p, and there is multiplying by constant, so its running time is Logp

Inner Loops running time = Logp x Logn – Loglogn

For outer loop, inner loops run n times so n x loglogn

So closest Time Complexity of the algorithm is O(nloglogn)

c)

Solution:

Suppose m = 20 and n =2 then

m=m-n = 20, 18, 16, 14, 12, 10, 8, 6, 4, 2, now both are equal ----- $n/2 = O(n)$

Now suppose m= 10 and n =1;

$M=m-n = 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 = O(n)$

Now Suppose $m= 20$ and $n = 3$;

$M=m-n = 20, 17, 14, 11, 8, 5, 2$, Now here n is greater, so $n=n-m, 3, 1$, again m is greater so $m=m-n, 2, 1 \Rightarrow n/3 + 2 + 1 = O(n)$

d)

Solution:

Here there is Independent Nested Loop

So the total frequency of $k=K+n/2$ will be same as the total frequency of nested loop

Outer Loop Frequency

$l=n/2, n/2+1, n/2+2, \dots, n/2+k$

Assume $(n/2)+k = n \Rightarrow k = n/2$ [$n/2 + k = n \Rightarrow [i.e (n+2k)/2 = n \Rightarrow n+2k = 2n \Rightarrow k = n/2]$]

Inner Loop frequency

$J=2, 2^2, 2^3, \dots, 2^k$ Assume $2^k = n \Rightarrow$ Log on both Side , $k=\log n$

So Algorithm time Complexity will be $n \times \log n \Rightarrow O(n \log n)$

e)

Solution :

There is independent Nested Loop

outer loop is running n times - $O(n)$

Inner loop

inner loop is increment by $j \times k$ and K is multiplying $k \times 2$ in each iteration of outer loop.

I	J	k
0	$J=0, \dots, n$	1
1	$J=0 \dots n/2$	2
2	$J=0 \dots n/4$	4
	.	
	.	
	.	
n	$J=0 \dots n/2^{i-1}$	
the doubling of k makes the total runtime linear in n.	$=n+n/2+n/4 \dots n/2^{i-1}$ Taking n common..its a geometric series $=n(1+1/2+1/4, \dots) \Rightarrow 2n$	Note: after some outer loop iterations, $k > n$, so from this point inner loop runs 1 time, but it does affect time complexity

	$= O(n)$	
--	----------	--

5. Prove that

$$5n^2 - 100n + 50 \in O(n^2)$$

$$n^4 + 50n^3 \notin O(n^3)$$

$$n^2 + n \log n \in O(n^2)$$

$$n(\log n)^2 + n \log n \in O(n(\log n)^2)$$

$$4n^2 - 1000n + 25 \in \Omega(n^2)$$

$$n^2 + n \log n \in \Omega(n^2)$$

$$\log n \notin \Omega(n).$$

$$10n^2 - 200n + 500 \in \Theta(n^2)$$

$$n^2 + n \log n \in \Theta(n^2)$$

$$n \log n + 50 \in \Theta(n \log n)$$

Big O, Omega, and Theta Notation Proofs

Definitions

Big O Notation: $f(n) \in O(g(n))$ if $\exists$ positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

Big Omega Notation: $f(n) \in \Omega(g(n))$ if $\exists$ positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

Big Theta Notation: $f(n) \in \Theta(g(n))$ if $f(n) \in O(g(n))$ AND $f(n) \in \Omega(g(n))$

Big O Proofs (Upper Bound)

1. Prove: $5n^2 - 100n + 50 \in O(n^2)$

Solution

Proof:

We need to find constants $c > 0$ and $n_0 > 0$ such that: $5n^2 - 100n + 50 \leq c \cdot n^2$ for all $n \geq n_0$

Let's rearrange this inequality: $5n^2 - 100n + 50 \leq c \cdot n^2$
 $5n^2 - 100n + 50 - c \cdot n^2 \leq 0$
 $(5 - c)n^2 - 100n + 50 \leq 0$

For large values of n , we want the coefficient of n^2 to be negative or zero, so we need: $5 - c \leq 0$, which means $c \geq 5$

Let's choose $c = 6$. Then we need: $5n^2 - 100n + 50 \leq 6n^2$
 $-n^2 - 100n + 50 \leq 0$
 $n^2 + 100n - 50 \geq 0$

To find when this inequality holds, we solve $n^2 + 100n - 50 = 0$: Using the quadratic formula: $n = \frac{-100 \pm \sqrt{(100)^2 + 200}}{2} = \frac{-100 \pm \sqrt{10200}}{2}$

Since $\sqrt{10200} \approx 101$, we get: $n \approx \frac{-100 + 101}{2} \approx 0.5$ or $n \approx \frac{-100 - 101}{2} \approx -100.5$

Since we need $n \geq n_0 > 0$, and the parabola $n^2 + 100n - 50$ opens upward, the inequality $n^2 + 100n - 50 \geq 0$ holds for $n \geq 0.5$.

Therefore, we can choose $n_0 = 1$.

Verification: For $n \geq 1$ and $c = 6$: $5n^2 - 100n + 50 \leq 6n^2$

Let's check this holds for $n = 1$: $5(1)^2 - 100(1) + 50 = 5 - 100 + 50 = -45$
 $6(1)^2 = 6$ Since $-45 \leq 6$

Conclusion: With $c = 6$ and $n_0 = 1$, we have shown that $5n^2 - 100n + 50 \leq 6n^2$ for all $n \geq 1$.

Therefore, $5n^2 - 100n + 50 \in O(n^2)$

2. Prove: $n^2 + n \log n \in O(n^2)$

Solution

Proof:

We need to find constants $c > 0$ and $n_0 > 0$ such that: $n^2 + n \log n \leq c \cdot n^2$ for all $n \geq n_0$

Rearranging the inequality: $n^2 + n \log n \leq c \cdot n^2$
 $n \log n \leq c \cdot n^2 - n^2$
 $n \log n \leq (c - 1)n^2$
 $\log n \leq (c - 1)n$ (dividing by n , since $n > 0$)

For this inequality to hold for large n , we need to choose c such that $\log n$ grows slower than $(c - 1)n$.

Since $\log n = o(n)$ (logarithmic functions grow slower than linear functions), we know that for any positive constant k , there exists some n_0 such that $\log n \leq k \cdot n$ for all $n \geq n_0$.

Let's choose $c = 2$, so we need: $\log n \leq (2 - 1)n = n$

We know that $\log n \leq n$ for all $n \geq 1$ (this is a well-known inequality).

To verify this more rigorously, consider that for $n \geq 2$:

- $\log_2 n \leq n$ (since 2^n grows much faster than n)
- If we're using natural logarithm, $\ln n \leq n$ for $n \geq 1$

Verification: For $n \geq 2$ and $c = 2$: $n^2 + n \log n \leq 2n^2$
 $n \log n \leq n^2$
 $\log n \leq n$

Let's check specific values:

- For $n = 2$: $2^2 + 2 \cdot \log 2 = 4 + 2 \cdot \log 2 \approx 4 + 1.39 = 5.39 \leq 2(4) = 8$
- For $n = 10$: $10^2 + 10 \cdot \log 10 = 100 + 10 \cdot \log 10 \approx 100 + 23 = 123 \leq 2(100) = 200$

Conclusion: With $c = 2$ and $n_0 = 2$, we have shown that $n^2 + n \log n \leq 2n^2$ for all $n \geq 2$.

Therefore, $n^2 + n \log n \in O(n^2)$

3. Prove: $n(\log n)^2 + n \log n \in O(n(\log n)^2)$

Solution

Proof:

We need to find constants $c > 0$ and $n_0 > 0$ such that: $n(\log n)^2 + n \log n \leq c \cdot n(\log n)^2$ for all $n \geq n_0$

Factor out n from both sides: $n[(\log n)^2 + \log n] \leq c \cdot n(\log n)^2$

Since $n > 0$ for $n \geq n_0$, we can divide both sides by n : $(\log n)^2 + \log n \leq c(\log n)^2$

Rearranging: $(\log n)^2 + \log n \leq c(\log n)^2 \implies \log n \leq c(\log n)^2 - (\log n)^2 \implies \log n \leq (c - 1)(\log n)^2$

Dividing both sides by $(\log n)^2$ (valid for $n \geq 2$ since $\log n > 0$): $\log n / (\log n)^2 \leq c - 1 \implies 1/\log n \leq c - 1$

Since $\log n$ grows without bound as $n \rightarrow \infty$, we have $1/\log n \rightarrow 0$ as $n \rightarrow \infty$.

This means that for any positive constant k , there exists some n_0 such that $1/\log n \leq k$ for all $n \geq n_0$.

Let's choose $c = 2$, so we need: $1/\log n \leq 2 - 1 = 1 \implies \log n \geq 1$

This inequality holds for $n \geq e \approx 2.718$. Since we typically work with integers, let's use $n_0 = 3$.

Verification: For $n \geq 3$ and $c = 2$: $n(\log n)^2 + n \log n \leq 2n(\log n)^2$

Dividing by $n(\log n)^2$: $1 + 1/\log n \leq 2$

For $n = 3$: $1 + 1/\log 3 \approx 1 + 1/1.1 \approx 1 + 0.91 = 1.91 \leq 2$ For $n = 10$: $1 + 1/\log 10 \approx 1 + 1/2.3 \approx 1 + 0.43 = 1.43 \leq 2$

As n increases, $1/\log n$ approaches 0, so $1 + 1/\log n$ approaches 1, which is certainly ≤ 2 .

Alternative approach: We can also factor the original expression: $n(\log n)^2 + n \log n = n \log n [(\log n) + 1] = n \log n \cdot (\log n + 1)$

Since $\log n + 1 \leq 2 \log n$ for $n \geq 2$ (because $\log n \geq 1$ when $n \geq e$): $n \log n \cdot (\log n + 1) \leq n \log n \cdot 2 \log n = 2n(\log n)^2$

Conclusion: With $c = 2$ and $n_0 = 3$, we have shown that $n(\log n)^2 + n \log n \leq 2n(\log n)^2$ for all $n \geq 3$.

Therefore, $n(\log n)^2 + n \log n \in O(n(\log n)^2)$

4. Prove: $n^4 + 50n^3 \notin O(n^3)$

Solution

Proof by contradiction:

Assume, for the sake of contradiction, that $n^4 + 50n^3 \in O(n^3)$.

Then there must exist positive constants c and n_0 such that: $n^4 + 50n^3 \leq c \cdot n^3$ for all $n \geq n_0$

Dividing both sides by n^3 (valid since $n > 0$ for $n \geq n_0$): $(n^4 + 50n^3)/n^3 \leq c \cdot n + 50 \leq c$

This inequality states that $n + 50 \leq c$ for all $n \geq n_0$.

However, since n can be arbitrarily large, $n + 50$ can also be arbitrarily large. No matter what positive constant c we choose, we can always find an n large enough such that $n + 50 > c$.

Specifically:

- If $c = 1000$, choose $n = 1000$, then $n + 50 = 1050 > 1000$
- If $c = 10^6$, choose $n = 10^6$, then $n + 50 = 10^6 + 50 > 10^6$
- In general, for any $c > 0$, choose $n = c + 1$, then $n + 50 = c + 51 > c$

This contradicts our assumption that such a constant c exists.

Alternative approach using limits:

Consider the limit: $\lim_{n \rightarrow \infty} (n^4 + 50n^3)/n^3 = \lim_{n \rightarrow \infty} (n + 50) = \infty$

Since this limit is infinite, there cannot exist a finite constant c such that $n^4 + 50n^3 \leq c \cdot n^3$ for all sufficiently large n .

Conclusion: Since we've reached a contradiction, our assumption was false.

Therefore, $n^4 + 50n^3 \notin O(n^3)$

Big Omega Proofs (Lower Bound)

5. Prove: $4n^2 - 1000n + 25 \in \Omega(n^2)$

Solution

Proof:

We need to find constants $c > 0$ and $n_0 > 0$ such that: $4n^2 - 1000n + 25 \geq c \cdot n^2$ for all $n \geq n_0$

Rearranging the inequality: $4n^2 - 1000n + 25 \geq c \cdot n^2$
 $4n^2 - 1000n + 25 - c \cdot n^2 \geq 0$
 $(4 - c)n^2 - 1000n + 25 \geq 0$

Let's choose $c = 2$, so we need: $(4 - 2)n^2 - 1000n + 25 \geq 0$ $2n^2 - 1000n + 25 \geq 0$

To find when this inequality holds, we solve $2n^2 - 1000n + 25 = 0$: Using the quadratic formula: $n = (1000 \pm \sqrt{(1000)^2 - 4 \cdot 2 \cdot 25})/4$ $n = (1000 \pm \sqrt{(1000000 - 200)})/4 = (1000 \pm \sqrt{999800})/4$

Since $\sqrt{999800} \approx 999.9$: $n \approx (1000 + 999.9)/4 \approx 499.975$ or $n \approx (1000 - 999.9)/4 \approx 0.025$

Since the coefficient of n^2 is positive ($2 > 0$), this parabola opens upward. Therefore: $2n^2 - 1000n + 25 \geq 0$ when $n \leq 0.025$ or $n \geq 499.975$

Since we need $n_0 > 0$ and we want the inequality to hold for all $n \geq n_0$, we choose $n_0 = 500$.

Verification: For $n \geq 500$ and $c = 2$: $4n^2 - 1000n + 25 \geq 2n^2$

Let's check for $n = 500$: $4(500)^2 - 1000(500) + 25 = 1,000,000 - 500,000 + 25 = 500,025$
 $2(500)^2 = 500,000$ Since $500,025 \geq 500,000$

For larger n , the n^2 term dominates and the inequality becomes even more clearly satisfied.

Alternative verification using limits: For large n , we can show that the ratio approaches a positive constant: $\lim_{n \rightarrow \infty} (4n^2 - 1000n + 25)/n^2 = \lim_{n \rightarrow \infty} (4 - 1000/n + 25/n^2) = 4$

Since this limit equals $4 > 0$, we can choose any $c < 4$ and find an appropriate n_0 .

Conclusion: With $c = 2$ and $n_0 = 500$, we have shown that $4n^2 - 1000n + 25 \geq 2n^2$ for all $n \geq 500$.

Therefore, $4n^2 - 1000n + 25 \in \Omega(n^2)$

6. Prove: $n^2 + n \log n \in \Omega(n^2)$

Solution

Proof:

We need to find constants $c > 0$ and $n_0 > 0$ such that: $n^2 + n \log n \geq c \cdot n^2$ for all $n \geq n_0$

Since $n \log n \geq 0$ for all $n \geq 1$ (because $n > 0$ and $\log n \geq 0$ for $n \geq 1$), we have: $n^2 + n \log n \geq n^2$ for all $n \geq 1$

This means we can choose $c = 1$ and $n_0 = 1$.

Verification: For $n \geq 1$ and $c = 1$: $n^2 + n \log n \geq 1 \cdot n^2$

This is clearly true since we're adding a non-negative term ($n \log n$) to n^2 .

Let's check specific values:

- For $n = 1$: $1^2 + 1 \cdot \log 1 = 1 + 0 = 1 \geq 1 \cdot 1^2 = 1$
- For $n = 2$: $2^2 + 2 \cdot \log 2 = 4 + 2 \cdot \log 2 \approx 4 + 1.39 = 5.39 \geq 1 \cdot 2^2 = 4$
- For $n = 10$: $10^2 + 10 \cdot \log 10 = 100 + 10 \cdot \log 10 \approx 100 + 23 = 123 \geq 1 \cdot 10^2 = 100$

Alternative approach using limits: Consider the limit: $\lim_{n \rightarrow \infty} (n^2 + n \log n)/n^2 = \lim_{n \rightarrow \infty} (1 + \log n/n) = 1 + 0 = 1$

Since this limit equals $1 > 0$, we can choose any $c < 1$ (or $c = 1$) and the inequality will hold for sufficiently large n .

Even stronger result: We can actually choose a larger constant. Since $n \log n > 0$ for $n \geq 2$, we have: $n^2 + n \log n > n^2$

This means we could choose any $c < 1$, but $c = 1$ is the most natural choice and gives us the tightest bound.

Conclusion: With $c = 1$ and $n_0 = 1$, we have shown that $n^2 + n \log n \geq 1 \cdot n^2$ for all $n \geq 1$.

Therefore, $n^2 + n \log n \in \Omega(n^2)$

7. Prove: $\log n \notin \Omega(n)$

Solution

Proof by contradiction:

Assume, for the sake of contradiction, that $\log n \in \Omega(n)$.

Then there must exist positive constants c and n_0 such that: $\log n \geq c \cdot n$ for all $n \geq n_0$

Rearranging: $\log n/n \geq c$ for all $n \geq n_0$

This means the ratio $\log n/n$ must be bounded below by some positive constant c .

However, let's examine what happens to this ratio as n grows:

Using limits: $\lim_{n \rightarrow \infty} (\log n)/n = 0$

This is a fundamental limit that can be proven using L'Hôpital's rule: $\lim_{n \rightarrow \infty} (\log n)/n = \lim_{n \rightarrow \infty} (1/n)/1 = \lim_{n \rightarrow \infty} 1/n = 0$

Since the limit is 0, for any positive constant $c > 0$, no matter how small, there exists some N such that $\log n/n < c$ for all $n > N$.

Concrete example: Let's say someone claims $c = 0.01$ works. Then we need $\log n \geq 0.01n$ for all $n \geq n_0$.

But consider $n = 1000$:

- $\log_2(1000) \approx 9.97$
- $0.01 \times 1000 = 10$
- Since $9.97 < 10$, the inequality fails.

For $n = 10,000$:

- $\log_2(10,000) \approx 13.29$
- $0.01 \times 10,000 = 100$
- Since $13.29 < 100$, the inequality fails even more dramatically.

General argument: For any $c > 0$, we can find n large enough such that $\log n < c \cdot n$ fails to hold.

Specifically, we need $\log n < c \cdot n$, or equivalently, $\log n/n < c$.

Since $\lim_{n \rightarrow \infty} (\log n)/n = 0$, we can make $\log n/n$ arbitrarily close to 0 by choosing n large enough. This means we can always find an n such that $\log n/n < c$, contradicting the requirement that $\log n \geq c \cdot n$.

Conclusion: Since we've reached a contradiction for any choice of positive constant c , our assumption was false.

Therefore, **$\log n \notin \Omega(n)$**

Big Theta Proofs (Tight Bound)

8. Prove: $10n^2 - 200n + 500 \in \Theta(n^2)$

Solution

To prove: $f(n) \in \Theta(n^2)$

I need to prove both parts:

Part 1: Prove $10n^2 - 200n + 500 \in O(n^2)$

We need constants $c_1 > 0$ and $n_1 > 0$ such that: $10n^2 - 200n + 500 \leq c_1 \cdot n^2$ for all $n \geq n_1$

Rearranging: $10n^2 - 200n + 500 \leq c_1 \cdot n^2$ $(10 - c_1)n^2 - 200n + 500 \leq 0$

Choose $c_1 = 11$, so we need: $(10 - 11)n^2 - 200n + 500 \leq 0$ $-n^2 - 200n + 500 \leq 0$ $n^2 + 200n - 500 \geq 0$

Using the quadratic formula: $n = (-200 \pm \sqrt{(40000 + 2000)})/2 = (-200 \pm \sqrt{42000})/2$

Since $\sqrt{42000} \approx 204.9$: $n \approx (-200 + 204.9)/2 \approx 2.45$ or $n \approx (-200 - 204.9)/2 \approx -202.45$

Since the parabola opens upward, $n^2 + 200n - 500 \geq 0$ for $n \geq 2.45$.

Choose $n_1 = 3$.

Part 2: Prove $10n^2 - 200n + 500 \in \Omega(n^2)$

We need constants $c_2 > 0$ and $n_2 > 0$ such that: $10n^2 - 200n + 500 \geq c_2 \cdot n^2$ for all $n \geq n_2$

Rearranging: $10n^2 - 200n + 500 \geq c_2 \cdot n^2$ $(10 - c_2)n^2 - 200n + 500 \geq 0$

Choose $c_2 = 5$, so we need: $(10 - 5)n^2 - 200n + 500 \geq 0$ $5n^2 - 200n + 500 \geq 0$ $n^2 - 40n + 100 \geq 0$

Using the quadratic formula: $n = (40 \pm \sqrt{(1600 - 400)})/2 = (40 \pm \sqrt{1200})/2$

Since $\sqrt{1200} \approx 34.64$: $n \approx (40 + 34.64)/2 \approx 37.32$ or $n \approx (40 - 34.64)/2 \approx 2.68$

Since the parabola opens upward, $n^2 - 40n + 100 \geq 0$ for $n \leq 2.68$ or $n \geq 37.32$.

Choose $n_2 = 38$.

Verification:

For $n = 38$, $c_1 = 11$, $c_2 = 5$:

- $f(38) = 10(38)^2 - 200(38) + 500 = 14,440 - 7,600 + 500 = 7,340$
- $c_2 \cdot n^2 = 5(38)^2 = 7,220$
- $c_1 \cdot n^2 = 11(38)^2 = 15,884$

Check: $7,220 \leq 7,340 \leq 15,884$

Alternative approach using limits: $\lim_{n \rightarrow \infty} (10n^2 - 200n + 500)/n^2 = \lim_{n \rightarrow \infty} (10 - 200/n + 500/n^2) = 10$

Since this limit equals $10 > 0$, we can choose:

- Any $c_1 > 10$ for the upper bound

- Any $c_2 < 10$ for the lower bound

Conclusion:

- With $c_1 = 11$ and $n_1 = 3$: $10n^2 - 200n + 500 \in O(n^2)$
- With $c_2 = 5$ and $n_2 = 38$: $10n^2 - 200n + 500 \in \Omega(n^2)$

Therefore, **$10n^2 - 200n + 500 \in \Theta(n^2)$**

9. Prove: $n^2 + n \log n \in \Theta(n^2)$

Solution

I'll prove that $n^2 + n \log n \in \Theta(n^2)$ using the formal definition of Big Theta notation.

Definition: $f(n) \in \Theta(g(n))$ if and only if $f(n) \in O(g(n))$ AND $f(n) \in \Omega(g(n))$.

Given: $f(n) = n^2 + n \log n$, $g(n) = n^2$

To prove: $f(n) \in \Theta(n^2)$

I need to prove both parts:

Part 1: Prove $n^2 + n \log n \in O(n^2)$

We need constants $c_1 > 0$ and $n_1 > 0$ such that: $n^2 + n \log n \leq c_1 \cdot n^2$ for all $n \geq n_1$

Rearranging: $n^2 + n \log n \leq c_1 \cdot n^2 \implies n \log n \leq (c_1 - 1)n^2 \implies \log n \leq (c_1 - 1)n$ (dividing by n , since $n > 0$)

Since $\log n$ grows slower than any linear function, we know that $\log n \leq n$ for all $n \geq 2$.

Choose $c_1 = 2$, so we need: $\log n \leq (2 - 1)n = n$

This holds for all $n \geq 2$, so we can choose $n_1 = 2$.

Part 2: Prove $n^2 + n \log n \in \Omega(n^2)$

We need constants $c_2 > 0$ and $n_2 > 0$ such that: $n^2 + n \log n \geq c_2 \cdot n^2$ for all $n \geq n_2$

Since $n \log n \geq 0$ for all $n \geq 1$ (because $n > 0$ and $\log n \geq 0$ for $n \geq 1$), we have: $n^2 + n \log n \geq n^2$ for all $n \geq 1$

This means we can choose $c_2 = 1$ and $n_2 = 1$.

Verification:

For $n \geq 2$, $c_1 = 2$, $c_2 = 1$: $c_2 \cdot n^2 \leq n^2 + n \log n \leq c_1 \cdot n^2$ $1 \cdot n^2 \leq n^2 + n \log n \leq 2 \cdot n^2$

Let's check for $n = 10$:

- $f(10) = 10^2 + 10 \cdot \log 10 = 100 + 10 \cdot \log 10 \approx 100 + 23 = 123$
- $c_2 \cdot n^2 = 1 \cdot 100 = 100$
- $c_1 \cdot n^2 = 2 \cdot 100 = 200$

Check: $100 \leq 123 \leq 200$

Using limits for verification: $\lim_{n \rightarrow \infty} (n^2 + n \log n)/n^2 = \lim_{n \rightarrow \infty} (1 + \log n/n) = 1 + 0 = 1$

Since this limit equals $1 > 0$, we can choose:

- Any $c_1 > 1$ for the upper bound
- Any $c_2 < 1$ for the lower bound

Our choices of $c_1 = 2$ and $c_2 = 1$ satisfy these conditions.

Conclusion:

- With $c_1 = 2$ and $n_1 = 2$: $n^2 + n \log n \in O(n^2)$
- With $c_2 = 1$ and $n_2 = 1$: $n^2 + n \log n \in \Omega(n^2)$

Therefore, $n^2 + n \log n \in \Theta(n^2)$

10. Prove: $n \log n + 50 \in \Theta(n \log n)$

Solution

To prove: $f(n) \in \Theta(n \log n)$

I need to prove both parts:

Part 1: Prove $n \log n + 50 \in O(n \log n)$

We need constants $c_1 > 0$ and $n_1 > 0$ such that: $n \log n + 50 \leq c_1 \cdot n \log n$ for all $n \geq n_1$

Rearranging: $n \log n + 50 \leq c_1 \cdot n \log n$ $50 \leq (c_1 - 1) \cdot n \log n$ $50/(n \log n) \leq c_1 - 1$

Since $\lim_{n \rightarrow \infty} 50/(n \log n) = 0$, for any positive constant k , there exists some n_1 such that $50/(n \log n) \leq k$ for all $n \geq n_1$.

Choose $c_1 = 2$, so we need: $50/(n \log n) \leq 2 - 1 = 1 \Rightarrow 50 \leq n \log n$

For $n = 10$: $50 \leq 10 \cdot \log 10 \approx 10 \cdot 2.3 = 23$? No. For $n = 20$: $50 \leq 20 \cdot \log 20 \approx 20 \cdot 3.0 = 60$? Yes.

Let's verify $n_1 = 20$ works: For $n \geq 20$, we have $n \log n \geq 20 \cdot \log 20 \approx 60 > 50$.

Therefore, choose $n_1 = 20$.

Part 2: Prove $n \log n + 50 \in \Omega(n \log n)$

We need constants $c_2 > 0$ and $n_2 > 0$ such that: $n \log n + 50 \geq c_2 \cdot n \log n$ for all $n \geq n_2$

Since $50 \geq 0$, we have: $n \log n + 50 \geq n \log n$ for all $n \geq 2$

This means we can choose $c_2 = 1$ and $n_2 = 2$.

Verification:

For $n \geq 20$, $c_1 = 2$, $c_2 = 1$: $c_2 \cdot n \log n \leq n \log n + 50 \leq c_1 \cdot n \log n \Rightarrow 1 \cdot n \log n \leq n \log n + 50 \leq 2 \cdot n \log n$

Let's check for $n = 20$:

- $f(20) = 20 \cdot \log 20 + 50 \approx 20 \cdot 3.0 + 50 = 60 + 50 = 110$
- $c_2 \cdot n \log n = 1 \cdot 60 = 60$
- $c_1 \cdot n \log n = 2 \cdot 60 = 120$

Check: $60 \leq 110 \leq 120$

For $n = 100$:

- $f(100) = 100 \cdot \log 100 + 50 \approx 100 \cdot 4.6 + 50 = 460 + 50 = 510$
- $c_2 \cdot n \log n = 1 \cdot 460 = 460$
- $c_1 \cdot n \log n = 2 \cdot 460 = 920$

Check: $460 \leq 510 \leq 920$

Using limits for verification: $\lim_{n \rightarrow \infty} (n \log n + 50)/(n \log n) = \lim_{n \rightarrow \infty} (1 + 50/(n \log n)) = 1 + 0 = 1$

Since this limit equals $1 > 0$, we can choose:

- Any $c_1 > 1$ for the upper bound
- Any $c_2 < 1$ for the lower bound (but $c_2 = 1$ works perfectly)

Conclusion:

- With $c_1 = 2$ and $n_1 = 20$: $n \log n + 50 \in O(n \log n)$
- With $c_2 = 1$ and $n_2 = 2$: $n \log n + 50 \in \Omega(n \log n)$

Therefore, **$n \log n + 50 \in \Theta(n \log n)$**